

Integrating A Dialogue Tree Based Turkish Chatbot into an Open Source Python Coding Editor

Turgay Tugay Bilgin
Department of Computer Engineering
Faculty of Engineering and Natural Sciences
Bursa Technical University
Bursa, Turkey
turgay.bilgin@btu.edu.tr
ORCID: 0000-0002-9245-5728

Erdem Yavuz
Department of Computer Engineering
Faculty of Engineering and Natural Sciences
Bursa Technical University
Bursa, Turkey
erdem.yavuz@btu.edu.tr
ORCID: 0000-0002-3159-2497

Abstract—In this study, a dialogue tree-based Turkish chatbot has been developed and integrated into the Pynar Python code editor in order to help the users with the questions they may ask while learning the python language. The Pynar Python Code editor is developed as a TÜBİTAK supported open source project. The architecture we propose has been developed in such a way that it can respond to users' questions and give emotional feedback through dialogs and emojis. It also runs completely offline, without being connected to any internet site or web service. The chatbot is coded using Python language and a Turkish NLP library and Qt GUI library. Considering the age and education levels of the target user group of the Pynar editor, our chatbot will provide great convenience to those who are trying to learn Python programming on their own, with the possibility of asking questions and getting answers in their native language.

Keywords—Chatbots, Dialogue Trees, Coding Editors

I. INTRODUCTION

The term Chatbot consists of the words Chat (dialogue) and Bot (robot). Chatbot is a computer system that helps people interact with the computer through keyboard or voice recognition. Chatbots work on repetitive patterns without human intervention and simulate humans and access the prepared knowledge base and respond according to the matches of the questions asked [1]. Chatbots can also provide some services in addition to answering the question directly. For example, they can direct users to important information and offer interesting websites. Thus, users can find the information they are looking for with little effort [2].

II. CLASSIFICATION OF CHATBOTS

Table 1 shows the categories of chatbots. Some chatbots may belong to more than one category. A Chatbot can use a knowledge base according to the specified knowledge area. Chatbots that can answer any question fall into the general (generic) chatbot category. Chorus (the chatbot used in Google Hangouts, developed by Google in 2020) can be given as an example of general chatbots. The Guardian chatbot [3], those operating in more than one domain can be given as an example of an open domain chatbot. On the other hand, chatsbots such as InstuctableCrowd or SnapTravel, that can only answer questions related to a specific knowledge area, can be given as examples to close domain chatbot [4]. The PyNar chatbot developed within the scope of this project can also be classified as a closed domain chatbot, because it can only answer questions about the Python language and it correct the Python language coding errors.

TABLE I. CLASSIFICATION OF CHATBOTS [5].

Knowledge Domain	Generic
	Open Domain
	Close Domain
Service provided	Interpersonal
	Intrapersonal
	Inter-agent
Goals	Informative
	Chat based
	Task Based
Response Generation Method	Rule Based
	Retrieval based
	Generative
Human-aid	Human mediated
	Autonomous
Permissions	Open source
	Commercial
Communication Channel	Written
	Audio
	Visual

Inter-agent chatbots can also communicate with other chatbots. Alexa and Cortana are two chatbots integrated to communicate with each other [6]. If users use the chatbot to get certain information, these bots are called informative chatbots. Conversation-based chatbots engage in a natural conversation with the user, just like a real person would. Task-based chatbots are successful in requesting information, responding, and perform tasks such as room reservations [4, 6].

Chatbots are divided into three groups according to the response generation method. These are known as rule-based, retrieval-based, and generative-based chatbots[7]. Rule-based chatbots match user input with a rule pattern and select a predefined response from a set of responses using AIML (Artificial Intelligence Markup Language) [8].

In some cases where more flexibility is required, a chatbot may co-operate with a human. A human-mediated chatbot takes human assistance. The biggest disadvantage of these chatbots is that they lack fast information processing due to the limitation of manpower and cannot cope with the large number of user requests [4].

Chatbots may also categorized by communication channel they use. Communication channel may can be the plain text,

audio, image or all of these are. New generation chatbots can react to images and can also recognize objects in images, make comments about them and express their feelings [9].

III. STRUCTURE OF DIALOGUE TREES

Dialogue Tree, is a game mechanic used in adventure, action-adventure and role-playing video games. When the player interacts with a character in the game, the player is given a choice of what to say and next choices are made at each step until the conversation is over. Some video games are executed almost entirely through such character interactions and branching dialogues. Dialogue trees are also preferred in educational chatbots. A dialogue tree that teaches subtraction operation is given in Fig. 1.

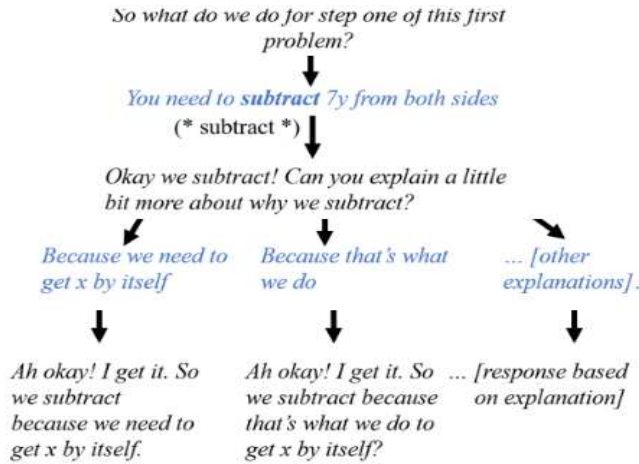


Fig. 1. An example of a dialogue tree that teaches subtraction [10].

A. Response Generation in Dialogue Tree Based Chatbots

As technology evolves, larger and more detailed virtual environments are created, more engaging NPCs (Non-player characters) are needed. Many games use some kind of dialogue tree that allows the user to choose their questions/answers while chatting with the characters in the game app. This method provides a dynamic conversation system.

Emotional response is one of the key factors in decision making and interactions between individuals [11, 12]. Including emotions in the dialogue will increase the impact of conversation. Therefore, emojis are also used in the chatbot conversations to strengthen the emotional effect in the dialogues. We also used emojis extensively in chatbot dialogs of our proposed chatbot engine.

Dialogue trees are hierarchical data structures that enable the progression of a speech scene based on user input. Typically, multiple answer options are provided for a question or answer to interact with users. Once an option is selected, the Chatbot will respond appropriately. According to the users' speech preferences, the dialogue tree is navigated and new options for the answer become available. There are two main methods of implementing such dialog trees: The first is the Hub and Spoke model, and the other is the Waterfall model [14].

B. Hub and Spoke Dialogue Tree Model

As seen in Fig. 2, the Hub and Spoke system has a central hub that will be divided into different speech options to facilitate dialogue interactions, each option initiates a different NPC response flow according to the user's choice [14]. After

all options are exhausted, the dialog tree will return to the central hub. The primary purpose of the Hub and Spoke system is to provide descriptive information to the user, it usually only ends when a desired option is selected or the user decides to end the conversation. This dialogue tree method results in a multi-formula conversational style in which users are presented with repetitive dialogues that do not mimic real-life interactions between two people.

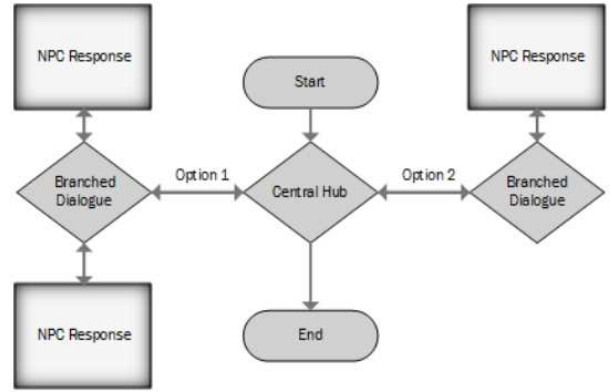


Fig. 2. Example of a hub and spoke dialog tree [14].

C. Waterfall Dialogue Tree Model

As can be seen in Fig. 3, the waterfall model always directs users to new options and does not allow them to return to the previous unselected selections. A particular selection can be expanded, but the overall flow remains the same, so there is a constant flow of new information.

While a dialogue tree following the waterfall technique provides a more realistic way of speaking than the Hub and Spoke system, it can be much more restrictive for the narration of a subject. Depending on the options the user chooses, it can lead to a very artificial conversation where the NPC's response does not match the questions the user asks, or responds in an irrelevant way.

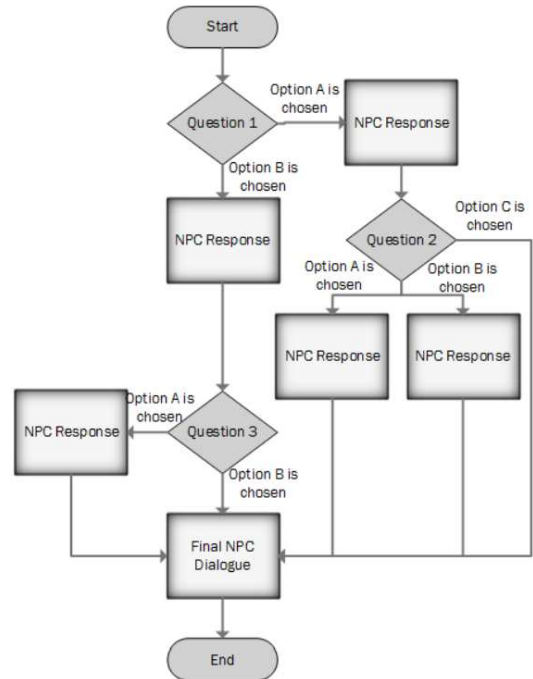


Fig. 3. Waterfall dialogue tree example [14].

Due to the limitations inherent in both models, many modern applications choose a mix of both Waterfall and Hub and Spoke dialogue trees [13]. This allows for the freedom of the Hub and Spoke model while taking advantage of the realism of the Waterfall method. Thanks to the combination of these methods, there is more consistency between the player's previous speech choices and how the NPC reacts, while ensuring realistic interaction between the player and the NPC.

IV. DIALOGUE TREE BASED PYNAR CHATBOT ARCHITECTURE

The chatbot developed within the scope of this study was developed for Pynar Editor [15], which is an open source Python editor developed by our team. The front-end design of Pynar Chatbot has been prepared with Python programming language and PyQt5 visual interface library. A Dialog Tree structure, an example of which is shown in Fig. 4, is used for the chatbot in the Pynar application to interact with the users. In this tree structure, there is a root level at the top and as many branches as desired can be added under this level. Each branch represents an option, in other words, an answer that matches the user's search criteria. Depth can be added to the options as desired. Depths are subcategories of the corresponding answer and in the Pynar interface, users are given the option to move between these branches with buttons.

When the user enters a word or sentence related to the subject he/she is curious about, basic data preprocessing steps such as system Tokenization, Stop words removal, Normalization, Stemming are performed. The flow diagram of the system is given in Fig. 5. With the words obtained as a result of these stages, SQLite query sentences are created in the form that can be searched from the Chatbot database.

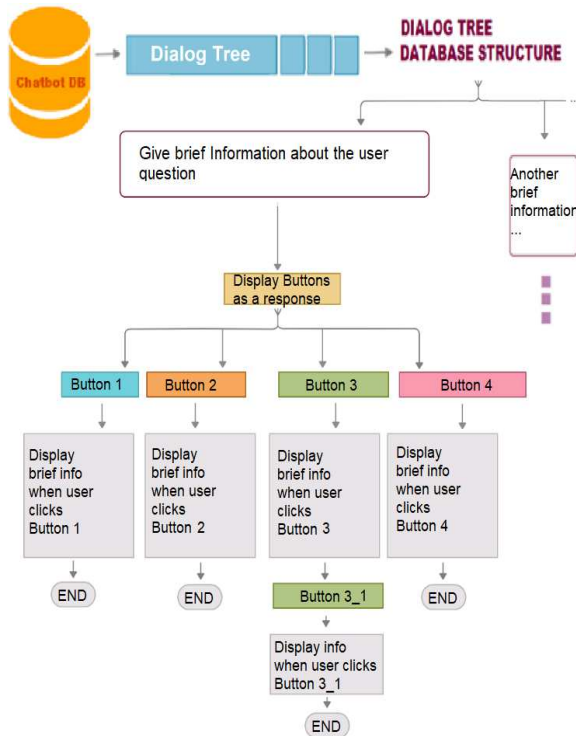


Fig. 4. Example of pynar dialog tree.

With these queries, searches are performed at all levels of the DialogTree table and returns the level and sub-branches it finds the answer in response to the relevant unit of the system. The system displays these answers in a format that the user

can see (Ex: Buttons, chat bubbles). The natural language processing activities performed at this stage are as follows:

- **Stop Words cleaning process:** Expressions that do not add any extra meaning to a text and do not change the meaning of the sentence if removed from the text. In Turkish, “and”, “with”, “ki” etc. or punctuation marks.

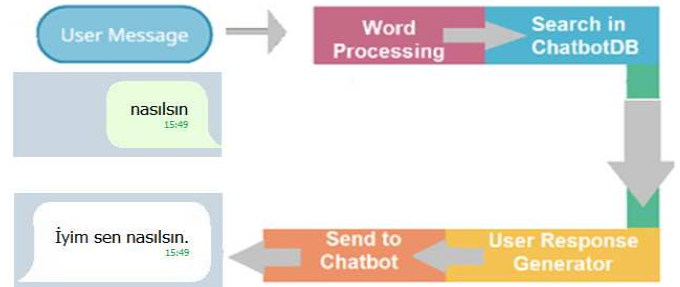


Fig. 5. Block diagram of pynar chatbot.

- **Tokenization:** When we ask the Pynar chatbot, “What does the for loop do?”, the system first splits this sentence into its tokens. The “word_token” function shown in Fig. 6 removes the stop words from the sentence and separates the sentence into its tokens. After tokenization, a 5-character string is obtained as seen in Fig. 6.,
- **Stemming:** Turkish natural language is an agglutinative language. It is different in structure from English and has different difficulties in terms of natural language processing. Finding the root of both the words “sarılık” (jaundice) and “sarılmak” (hug) as “sari” (yellow) will cause the comprehension process to be wrong. For this reason, correct stemming is very important for understanding natural language. In addition, in cases such as letter softening and hardening in the Turkish language, it is important when specifying the root of words.

```
>>> text
'For döngüsü ne işe yarar ?'
>>> word_token(text)
['For', 'döngüsü', 'ne', 'işe', 'yarar']
```

Fig. 6. Tokenization operations in the pynar chatbot.

```
['For', 'döngüsü', 'ne', 'işe', 'yarar']

1 2 3 4 5

['for', 'döngü', 'ne', 'iş', 'yarar']
```

Fig. 7. Stemming process in pynar chat.

As seen in Figure 7, the root of the word “döngüsü” is taken as “döngü” and the root of the word “işe” is taken as “iş”.

Since the other words are already in root form in the sentence, there was no change after the stemming process.

V. EXPERIMENTS AND DEMONSTRATION

In this section, the chatbot has been tested to get information about two different topics commonly used in Python, namely loops, functions and tuples, and the test results are given with screenshots.

A. Scenario 1: Asking about the “Loops”



Fig. 8. Example of answer for input “döngü nedir? (what is a loop?)” in the pynar chatbot.

In case the user asks the question “Döngü nedir?” (What is a loop?) to the chatbot, the dialog will be as in Fig. 8. First of all, the chatbot will give general information on the topic of “Loops”. Under this information message, there are buttons for sub-items that can be answered about “Loops”.

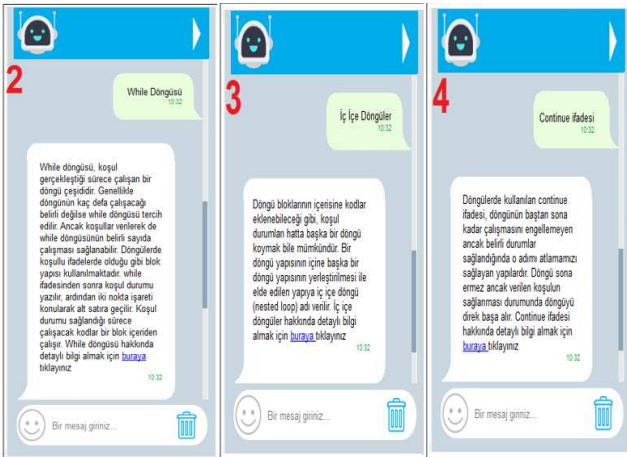


Fig. 9. Buttons 2, 3 and 4 answers to the input “döngü nedir?” in the pynar chatbot.

As it can be seen with the red oval in the figure, if the information provided by the chatbot is not sufficient, the offline HTML help document in the chatbot can be opened

and detailed information can be accessed. The details of the subcategories are shown in Fig. 9.

There are 2 levels for this topic in the dialog database, so the dialog ends after getting information about the sub-loops.

B. Scenario 2: Asking about the “Functions”

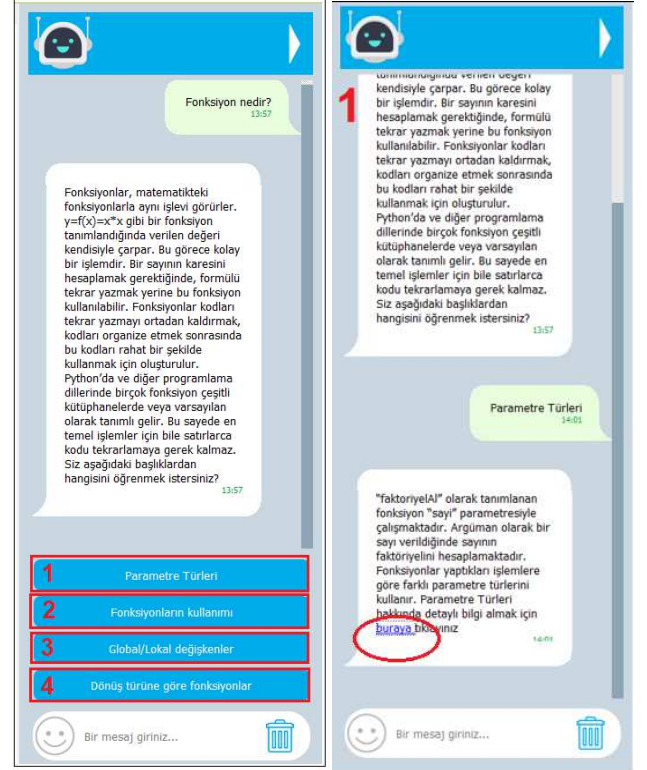


Fig. 10. Example of answer to input “fonksiyon nedir?” in the pynar chatbot.

If the user asks the question “Fonksiyon nedir?” (What is the function?) to the chatbot, the dialog will be as in Fig. 10. First of all, the chatbot will give general information about Functions. Under this information message, there are buttons for sub-items that can be answered about functions. Details of the subcategories are shown in Fig. 11.

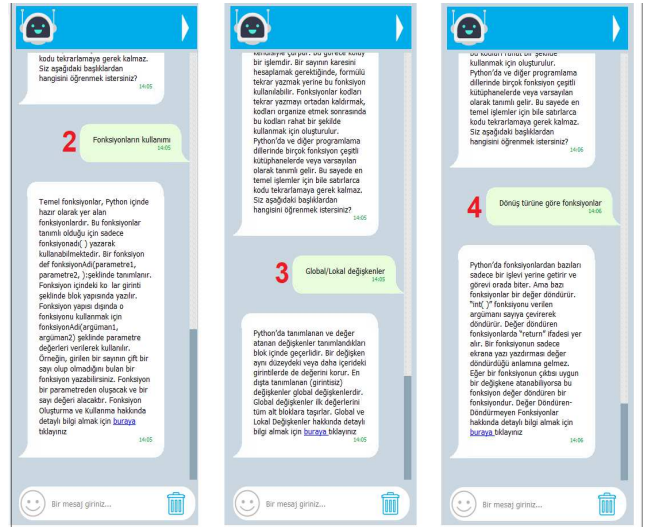


Fig. 11. Buttons 2, 3 and 4 answers to the input “fonksiyon nedir?” in the pynar chatbot

There are 2 levels for the example of functions in the dialog database, so the dialog ends after getting information about the sub-loops.

VI. CONCLUSION

This study has been developed to help users with questions they may ask while learning Python language within the Pynar Python code editor we have developed. Considering the age and education levels of the target user group of the Pynar editor, our chatbot will provide great convenience to those who are trying to learn Python programming on their own, with the possibility of asking questions and getting answers in their native language. The simplicity and systematicity of the information in the chatbot dialog tree database is suitable for the target audience to understand. Thanks to its user-friendly interface and its interactive structure with the user, it supports the active participation of people in the learning process.

VII. DISCUSSION

Other algorithms will be used instead of the dialogue tree model-based chatbot. A dialogue generating engine based on deep learning will be developed. In order to help the chatbot answer all python questions and increase its chances of success, we will also provide it internet access.

ACKNOWLEDGMENT

This study has been supported by TUBITAK (The Scientific and Technological Research Council of Turkey) under project number 118E882.

REFERENCES

- [1] A. Braun, *Chatbots in der Kundenkommunikation*. Springer-Verlag Berlin Heidelberg, 2003.
- [2] C. Möbus, *Web-Kommunikation mit OpenSource: Chatbots, Virtuelle Messen, Rich-Media-Content*. Springer-Verlag, 2006.
- [3] N. Good, and C. Wilk, "Introducing the guardian chatbot.", *The Guardian*, 2016, <https://www.theguardian.com/help/insideguardian/2016/nov/07/>, Access date: 15 June 2022.
- [4] P. Kucherbaev, A. Bozzon, and G. J. Houben, "Human-aided bots," *IEEE Internet Computing*, 22(6), pp. 36–43, <http://dx.doi.org/10.1109/MIC.2018.252095348>.
- [5] E. Adamopoulou and L. Moussiades, "Chatbots: History, technology, and applications," *Machine Learning with Applications*, 2, 100006, 2020.
- [6] K. Nimavat and T. Champaneria, "Chatbots: An overview types, architecture, tools and future possibilities," *International Journal for Scientific Research and Development*, 5(7), pp. 1019–102, (2017).
- [7] H. T. Hien, P. N. Cuong, L. N. H Nam, H. L. T. K. Nhung, and L. D Thang, "Intelligent assistants in higher-education environments: The FIT-EBot, a chatbot for administrative and learning support," In *Proceedings of the ninth international symposium on information and communication technology*, pp. 69–76. New York, NY, USA: ACM, 2018. <http://dx.doi.org/10.1145/3287921.3287937>.
- [8] M. D. G. B. Marietto, R. V. de Aguiar, G. D. O. Barbosa, W. T. Botelho, E. Pimentel, R. D. S. França, and V. L. da Silva, "Artificial intelligence markup language: a brief tutorial," *arXiv preprint arXiv:1307.3091*, 2013.
- [9] H. Shum, X. He, and D. Li, "From eliza to xiaoice: Challenges and opportunities with social chatbots," *Frontiers of Information Technology & Electronic Engineering*, 19(1), pp. 10–26, 2018. <http://dx.doi.org/10.1631/FITEE.1700826>.
- [10] N. Lubold, "Producing acoustic-prosodic entrainment in a robotic learning companion to build learner rapport," *Doctoral dissertation*, Arizona State University, 2018.
- [11] G. L. Clore and J. R. Huntsinger, "How emotions inform judgment and regulate thought," *Trends in cognitive sciences*, 11(9), pp. 393–399, 2007.
- [12] J. S. Lerner, Y. Li, P. Valdesolo, and K. Kassam, "Emotion and decision making," *Annual Review of Psychology*, 66, 2015.
- [13] A. Freed, "Branching Conversation Systems and the Working Writer, Part 2: Design Considerations," *GAMASUTRA, The Art & Business of Making Games*, 2014.
- [14] J. Collins, W. Hisrt, W. Tang, C. Luu,, P. Smith, A. Watson, and R. Sahandi, "EDTree: Emotional dialogue trees for game based training," In *International Conference on Technologies for E-Learning and Digital Entertainment*, pp. 77–84, Springer, Cham, 2016.
- Pynar Python Editor, 2022, <http://www.pynar.org> , Access date: 15 June 2022
- [15] Pynar Python Editor, 2022, <http://www.pynar.org> , Access date: 15 June 2022.